

Laporan Tugas Besar Milestone 2

**IF2224 Teori Bahasa Formal dan Otomata
Syntax Analysis
Semester II 2025/2026**



Disusun oleh:

Fauzan Mohamad Abdul Ghani	13524113
Safira Berlianti	13524128
Eduard Daniel Ariajaya	13524129
Muhammad Daffa Arrizki Yanma	13524133
Reysha Syafitri Mulya Ramadhan	13524137

**Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026**

Daftar Isi

BAB I

Teori Singkat.....	3
1.1. Definisi dan Peran Parser.....	3
1.2. Keterkaitan dengan Lexer.....	3
1.3. Context-Free Grammar.....	3
1.4. Top-Down Parsing.....	4
1.5. Algoritma Recursive Descent.....	4
1.6. Parse Tree.....	5

BAB II

Perancangan & Implementasi.....	7
2.1. Perancangan Program.....	7
2.1.1. Teknologi yang Digunakan.....	7
2.1.2. Struktur Program.....	7
2.1.3. Alur Kerja Program.....	7
2.1.4. Justifikasi Pilihan Implementasi.....	8
2.2. Implementasi.....	8
2.2.1. Struktur Data Token.....	8
2.2.2. Kelas Parser.....	8
2.2.3. Fungsi Bantuan.....	9
2.2.4. Hierarki Fungsi Parser.....	9
2.2.5. Penanganan Ekspresi.....	10
2.3. Aturan Grammar.....	10

BAB III

Pengujian.....	16
3.1. Test Case 1.....	16
3.2. Test Case 2.....	17
3.3. Test Case 3.....	19
3.4. Test Case 4.....	22
3.5. Test Case 5.....	25
3.6. Test Case 6 (Error Handling).....	28

BAB IV

Penutup.....	30
4.1. Kesimpulan.....	30
4.2. Saran.....	30

LAMPIRAN.....	31
----------------------	-----------

REFERENSI.....	33
-----------------------	-----------

BAB I

Teori Singkat

1.1. Definisi dan Peran Parser

Parser atau syntax analyzer merupakan tahap lanjutan setelah lexical analysis dalam proses kompilasi. Parser menerima masukan berupa token-token hasil lexer, kemudian memeriksa apakah urutan token tersebut sesuai dengan aturan grammar bahasa pemrograman yang digunakan.

Selain melakukan validasi sintaks, parser juga membangun representasi struktur program dalam bentuk Parse Tree atau Abstract Syntax Tree (AST). Struktur ini digunakan pada tahap berikutnya, seperti semantic analysis dan code generation.

Secara umum, parser memiliki beberapa peran utama:

- Memastikan struktur sintaks program valid sesuai grammar.
- Mendeteksi dan melaporkan syntax error.
- Membentuk representasi hierarkis program untuk proses kompilasi selanjutnya.

1.2. Keterkaitan dengan Lexer

Lexer dan parser merupakan dua tahap yang saling berkaitan dalam compiler. Lexer bertugas membaca source code berupa rangkaian karakter dan mengubahnya menjadi token-token seperti identifier, keyword, operator, maupun literal.

Token yang dihasilkan lexer kemudian menjadi masukan bagi parser. Parser tidak lagi memproses karakter secara langsung, melainkan memeriksa pola token untuk menentukan apakah susunan program sudah sesuai dengan grammar bahasa.

Lexer berfokus pada pengenalan bentuk token (lexical structure), sedangkan parser berfokus pada pemeriksaan struktur sintaks program (syntactic structure).

1.3. Context-Free Grammar

Context-Free Grammar (CFG) merupakan sekumpulan aturan produksi yang digunakan untuk mendefinisikan struktur sintaks suatu bahasa pemrograman. Dalam parser, grammar digunakan sebagai acuan untuk menentukan apakah susunan token membentuk program yang valid.

Grammar bahasa Arion terdiri atas:

- 1) Non-terminal, yaitu simbol yang masih dapat diturunkan menjadi bentuk lain, seperti `<program>` dan `<statement>`.
- 2) Terminal, yaitu token hasil lexical analysis, seperti `ident`, `intcon`, `beginsy`, dan `becomes`.
- 3) Start symbol, yaitu simbol awal parsing, pada grammar Arion berupa `<program>`.
- 4) Production rule, yaitu aturan pembentukan struktur sintaks, misalnya:
`<program> → <program-header> <declaration-part> <compound-statement> period`

1.4. Top-Down Parsing

Top-down parsing merupakan metode parsing yang membangun parse tree dimulai dari root atau start symbol, kemudian menurunkan aturan grammar hingga menghasilkan token-token terminal yang sesuai dengan input. Proses parsing dilakukan dari kiri ke kanan dengan mencoba mencocokkan token masukan terhadap aturan produksi grammar.

Pada pendekatan ini, parser melakukan derivasi dari simbol awal menuju bentuk akhir program menggunakan aturan produksi pada Context-Free Grammar (CFG). Jika token yang dibaca tidak sesuai dengan aturan yang diharapkan, parser akan menghasilkan syntax error.

Top-down parsing umumnya lebih mudah dipahami dan diimplementasikan dibandingkan bottom-up parsing karena struktur parsing mengikuti bentuk grammar secara langsung. Namun, metode top-down parsing memiliki beberapa keterbatasan, seperti kesulitan menangani grammar yang mengandung left recursion.

1.5. Algoritma Recursive Descent

Recursive Descent merupakan metode parsing top-down yang menggunakan sekumpulan fungsi rekursif untuk merepresentasikan aturan produksi pada grammar. Setiap non-terminal pada grammar diimplementasikan sebagai sebuah fungsi parser.

Proses parsing dimulai dari start symbol, kemudian parser akan menurunkan aturan grammar secara bertahap hingga seluruh token input berhasil dicocokkan. Jika token yang dibaca tidak sesuai dengan aturan yang diharapkan, parser akan menghasilkan syntax error.

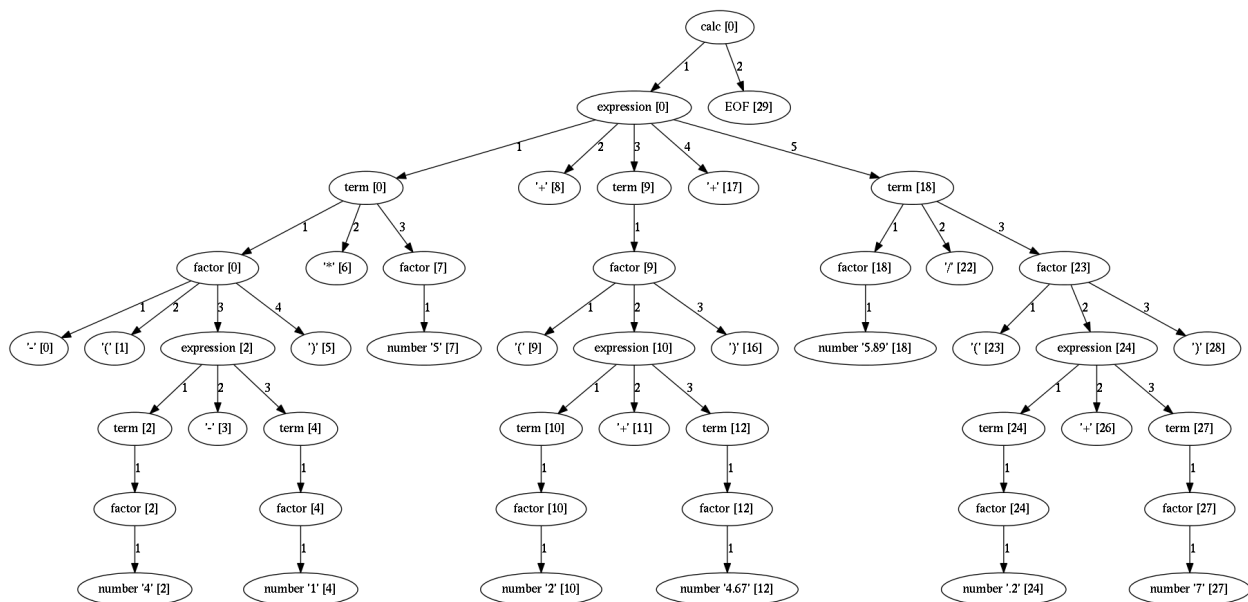
Langkah umum Recursive Descent:

- 1) Parsing dimulai dari start symbol grammar.
- 2) Parser memanggil fungsi sesuai non-terminal yang sedang diproses.
- 3) Token input dicocokkan dengan aturan produksi secara berurutan.
- 4) Jika seluruh token sesuai dengan grammar, parsing dinyatakan berhasil.

5) Jika terjadi ketidaksesuaian token, parser mengeluarkan syntax error.

1.6. Parse Tree

Parse Tree adalah representasi berbentuk pohon yang menggambarkan struktur sintaks program berdasarkan aturan grammar. Setiap node pada parse tree merepresentasikan simbol grammar, baik terminal maupun non-terminal.



Contoh Parse Tree

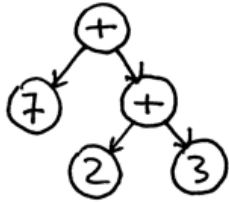
Sumber: https://textx.github.io/Arpeggio/latest/images/calc_parse_tree.dot.png

Parse tree menunjukkan bagaimana token-token hasil lexical analysis diturunkan dari start symbol hingga membentuk program yang valid. Struktur pohon ini membantu menggambarkan hubungan hierarkis antar bagian program, seperti ekspresi, statement, maupun deklarasi.

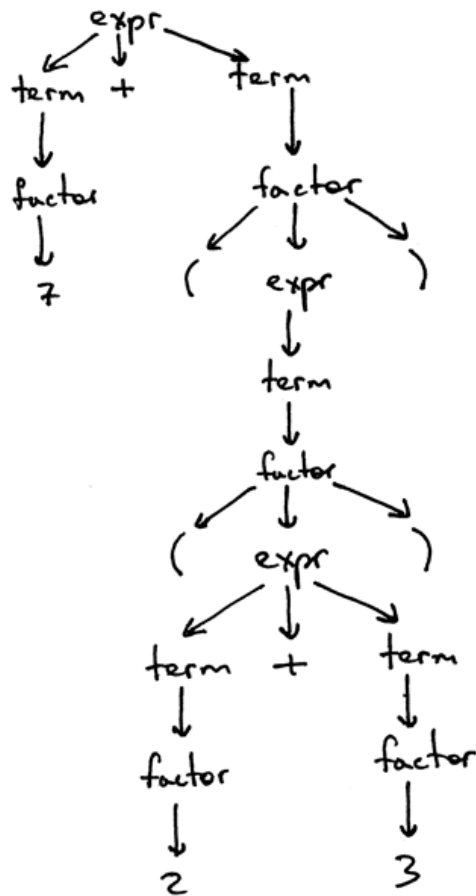
Dalam compiler, parse tree digunakan sebagai representasi awal struktur program sebelum diproses lebih lanjut pada tahap semantic analysis atau diubah menjadi AST yang lebih sederhana.

$$7 + ((2 + 3))$$

AST



Parse tree



Gambar Contoh Pembuatan Parse Tree dan Perbedaannya dengan AST (Abstract Syntax Tree)

(Sumber: <https://ruslanspivak.com/lsbasi-part7>)

BAB II

Perancangan & Implementasi

2.1. Perancangan Program

2.1.1. Teknologi yang Digunakan

- Program diimplementasikan menggunakan C++17 dengan compiler g++
- Build system menggunakan Makefile untuk kompilasi seluruh file .cpp di folder src/

2.1.2. Struktur Program

```
src/  
├──main.cpp  
├── ArionLexer.hpp / .cpp  
├── KeywordIdentifierLexer.hpp / .cpp  
├── TipeDataKomentar.hpp / .cpp  
├── OperatorDelimiterLexer.hpp / .cpp  
└── Parser.hpp / .cpp
```

2.1.3. Alur Kerja Program

- Lexer dijalankan.
ArionLexer membaca source code karakter per karakter menggunakan DFA, menghasilkan token satu per baris ke output_token.txt.
- ArionLexer di-destruct.
File token di-flush dan ditutup sebelum dibaca parser. Ini mencegah race condition antara penulisan dan pembacaan file.
- Token dimuat.
loadTokensFromFile() membaca output_token.txt, mengurai setiap baris menjadi struct Token { type, value }, melewati baris kosong dan token comment.
- Parser dijalankan.
Jika token pertama adalah programsy, parser membangun parse tree dengan Recursive Descent, mencetak output ke terminal dan output_tree.txt.

2.1.4. Justifikasi Pilihan Implementasi

Keputusan	Pilihan	Alasan
Algoritma parsing	Recursive Descent	Mudah dipetakan 1-ke-1 dari aturan grammar, mudah debug, cukup untuk grammar LL(1) bahasa Arion
Lookahead	1 token	Grammar Arion tidak memerlukan lebih dari 1 token untuk menentukan aturan produksi yang tepat
Error handling	throw std::runtime_error	Memungkinkan parser gagal tanpa exit() sehingga file output Lexer tetap tersimpan
Token comment	Dilewati di parser	Komentar bukan bagian dari struktur sintaks; cukup dikenali Lexer

2.2. Implementasi

2.2.1. Struktur Data Token

```
struct Token {  
    std::string type;  
    std::string value;  
};
```

Fungsi `loadTokensFromFile(string filename)`, membaca file token dengan pola type (value):

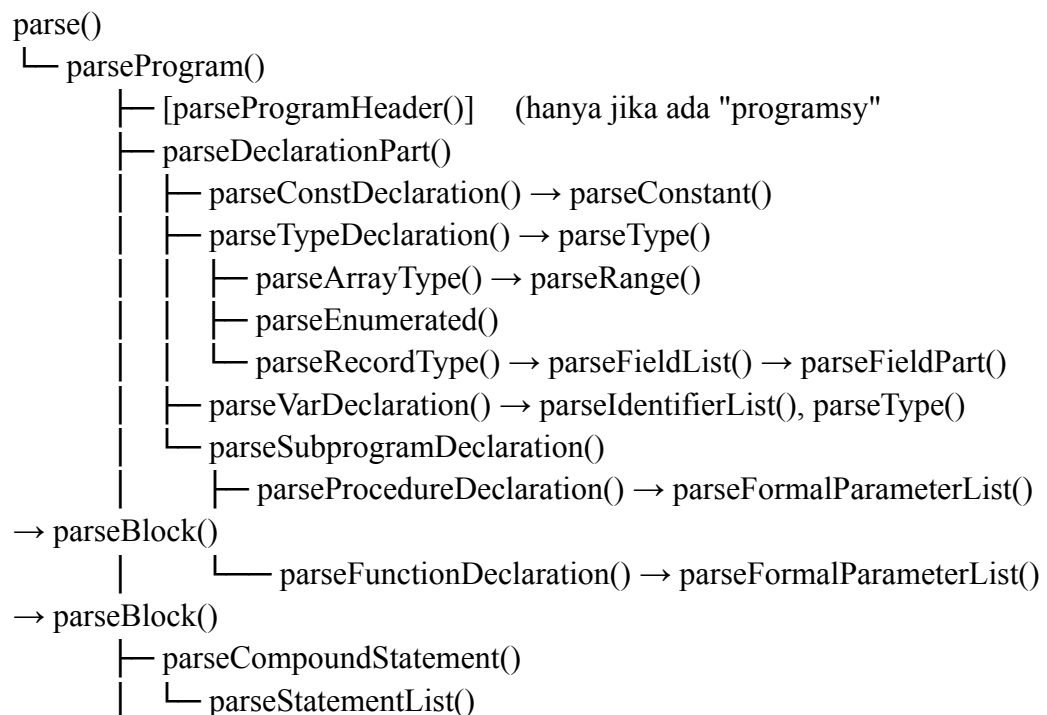
2.2.2. Kelas Parser

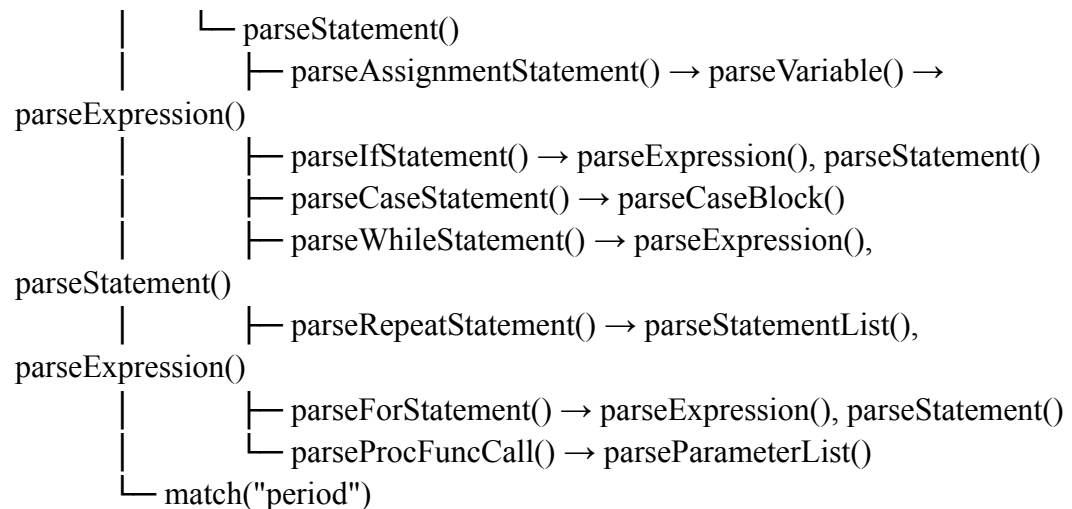
Atribut	Tipe	Peran
tokens	vector<Token>	Seluruh token dari Lexer
currentIndex	int	Posisi token yang sedang aktif
indentLevel	int	Kedalaman indentasi untuk cetak parse tree
outFile	ofstream	File tujuan output parse tree

2.2.3. Fungsi Bantuan

- `match(expectedType)`
Memproses token saat ini jika tipenya sesuai, lalu mencetak node terminal ke parse tree. Jika tidak cocok, melempar `std::runtime_error` dengan pesan informatif, contoh: "SYNTAX ERROR: token tidak terduga 'period', diharapkan 'semicolon'".
- `check(type)`
Melihat tipe token saat ini tanpa memprosesnya; digunakan untuk menentukan cabang grammar (if `check("ifsy") → parseIfStatement()`).
- `lookahead(offset)`
Melihat token ke depan sebanyak offset posisi tanpa memproses token tersebut, digunakan untuk membedakan:
 - `ident + lparent → pemanggilan fungsi/prosedur`
 - `ident + becomes → assignment statement`
 - `ident + lbrack/period → akses array/record`
- `printNode(nodeName)`
Mencetak satu simpul dengan indentasi | dan prefix |-- sesuai kedalaman

2.2.4. Hierarki Fungsi Parser





2.2.5. Penanganan Ekspresi

Hierarki ekspresi mengikuti prioritas operator dari rendah ke tinggi:

```

parseExpression()    → operator relasional (==, <, >, >=, <=, <=)
└─ parseSimpleExpression() → operator aditif (+, -, or)
    └─ parseTerm()          → operator multiplikatif (*, /, div, mod, and)
        └─ parseFactor()    → unit terkecil (literal, variabel, fungsi, (expr))

```

Struktur ini memastikan operator dengan prioritas lebih rendah berada di node yang lebih dekat ke root, sedangkan operator prioritas tinggi berada lebih dalam.

2.3. Aturan Grammar

No	Nama Node	Deskripsi	Aturan Produksi	Contoh
1	<program>	Node root yang merepresentasikan keseluruhan program Arion	program → program-header + declaration-part + compound-statement + period	Seluruh struktur program dari awal hingga akhir
2	<program-header>	Bagian kepala program yang berisi nama program	programsy + ident + semicolon	program Hello;

3	<declaration-part>	Bagian deklarasi yang dapat berisi const, type, var, procedure, function	(const-declaration)* + (type-declaration)* + (var-declaration)* + (subprogram-declaration)*	Semua deklarasi sebelum begin
4	<const-declaration>	Deklarasi konstanta	constsy + (ident + eq l + constant + semicolon) +	const INT == 100; REAL == 10.0; CHAR == 'a'; STRING == 'abcd';
5	<constant>	Konstanta dapat berupa ident , intcon , realcon , charcon , atau string	charcon string [(plus minus)? + (ident intcon realcon)]	'a' 'abcd' 10 +10.0 -a
6	<type-declaration>	Deklarasi tipe data baru	typesy + (ident + eq l + type + semicolon) +	type Range == 1..10; type Month == (Jan, Feb, Mar, Apr);
7	<var-declaration>	Deklarasi variabel	varsy + (identifier-list + colon + type + semicolon) +	var a, b: integer;
8	<identifier-list>	Daftar identifier yang dipisahkan koma	ident (comma + ident)*	a, b, c
9	<type>	Tipe data yang terdefinisi atau didefinisikan pengguna. Dapat berupa ident , array-type, range, atau enumerated	ident array-type range enumerated record-type	integer, array[1..10] of integer

10	<array-type>	Definisi tipe array	arraysy + lbrack + (range ident) + rbrack + ofsy + type	array[1..10] of integer array[char] of integer
11	<range>	Rentang nilai untuk array atau subrange	constant + period + period + constant	1..10, 'a'..'z'
12	<enumerated>	Nilai tertentu yang ada di dalam list	lparent + ident + (comma + ident)* + rparent	(Jan, Feb, Mar, Apr)
13	<record-type>	Tipe data template, sama dengan <i>class</i> dalam OOP	recordsy + field-list + endsy	type record-name = record field-1: field-type1; field-2: field-type2; ... field-n: field-typen; end
14	<field-list>	Penjabaran atribut/field dari record	field-part + (semicolon + field-part)*	..field-1.. ..field-2..
15	<field-part>	Singular atribut/field dari record	identifier-list + colon + type	field-1: field-type1;
16	<subprogram-declaration>	Deklarasi prosedur atau fungsi	procedure-declaration function-declaration	procedure print(x: integer);
17	<procedure-declaration>	Deklarasi prosedur	proceduresy + ident + (formal-parameter-list)? + semicolon + block + semicolon	Prosedur dengan parameter opsional

18	<function-declaration>	Deklarasi fungsi	function sy + ident + (formal-parameter-list)? + colon + ident + semicolon + block + semicolon	Fungsi yang mengembalikan nilai
19	<block>	Bagian kode yang mengandung deklarasi dan statement	declaration-part + compound-statement	var ... begin ... end
20	<formal-parameter-list>	Daftar parameter formal	lparent + parameter-group + (semicolon + parameter-group)* + rparent	(x, y: integer; z: real)
21	<parameter-group>	Daftar parameter disertai dengan tipe data parameter	identifier-list + colon + (ident array-type)	x, y: integer
22	<compound-statement>	Blok statement yang diawali begin dan diakhiri end	beginsy + statement-list + endsy	begin ... end
23	<statement-list>	Daftar statement yang dipisahkan semicolon	statement (semicolon + statement)*	Urutan statement dalam blok
24	<statement>	Deskripsi aksi yang harus dilakukan program komputer	(assignment-statement if-statement case-statement while-statement repeat-statement for-statement procedure/function-call)?	x := 5; if x > 0 then y := 1 else y := 0 while x < 10 do x := x + 1
25	<variable>	Variable dapat berbentuk ident, elemen array, atau field record	ident + (component-variable)*	someVar someArray[1] someField

26	<component-variable>	Variable dalam bentuk element array atau field record	(lbrack + index-list + rbrack) (period + ident)	someArray[1] someRecord.someField someArray[1][2]
27	<index-list>	Penunjuk index dalam array	(intcon charcon ident) + (comma + index-list)*	[1, 2, 3] ['a', 'b'] [true, false]
28	<assignment-statement>	Statement penugasan nilai ke variabel	variable + becomes + expression	x := 5, y := x + 10
29	<if-statement>	Statement kondisional	ifsy + expression + thensy + statement + (elsy + statement)?	if x > 0 then y := 1 else y := 0
30	<case-statement>	Statement kondisional dengan banyak kondisional	casesy + expression + ofsy + case-block + endsy	case i of 0: x := 0; 1 : x := 1; end
31	<case-block>	Deklarasi case dan statement yang harus dilakukan	constant + (comma + constant)* + colon + statement + (semicolon + case-block?)*	0: x := 0; 1, 2: x := 1
32	<while-statement>	Perulangan dengan kondisi di awal	whilesy + expression + dosy + statement	while x < 10 do x := x + 1
33	<repeat-statement>	Perulangan dengan kondisi di akhir	repeatsy + statement-list + untilsy + expression	repeat x := x - 1 until x == 0
34	<for-statement>	Perulangan dengan counter	forsy + ident + becomes + expression + (tosy downtosy) + expression + dosy + statement	for i := 1 to 10 do ...

35	<procedure/function-call>	Pemanggilan prosedur atau fungsi	ident + (lparent + parameter-list? + rparent)	writeln('Hello'), print(x, y)
36	<parameter-list>	Daftar parameter aktual saat pemanggilan	expression (comma + expression)*	'Result = ', b, 100
37	<expression>	Ekspresi yang menghasilkan nilai	simple-expression (relational-operator + simple-expression)?	x + y, a > b, 5 * (x + 1)
38	<simple-expression>	Ekspresi tanpa operator relasional	(plus minus)? term (additive-operator + term)*	x + y - z, 5 * 2
39	<term>	Bagian ekspresi dengan prioritas lebih tinggi	factor (multiplicative-operator + factor)*	x * y, a div b
40	<factor>	Unit terkecil dalam ekspresi	ident intcon realcon charcon string (lparent + expression + rparent) (notsy + factor) procedure/function-call variable	x, 5, 'a', (x+y), not a
41	<relational-operator>	Operator perbandingan	eq neq gtr geq lss leq	x == 5, y > 10
42	<additive-operator>	Operator penjumlahan/pengurangan	plus minus orsy	x + y, a or b
43	<multiplicative-operator>	Operator perkalian/pembagian	times rdiv idiv imod andsy	x * y, a div b, p and q

BAB III

Pengujian

3.1. Test Case 1

Input	Output
<pre> program Hello; var a, b: integer; begin a := 5; b := a + 10; writeln('Result = ', b); end.</pre>	<pre> <program> -- <program-header> -- programsy -- ident(Hello) -- semicolon -- <declaration-part> -- <var-declaration> -- varsy -- <identifier-list> -- ident(a) -- comma -- ident(b) -- colon -- <type> -- ident(integer) -- semicolon -- <compound-statement> -- beginsy -- <statement-list> -- <assignment-statement> -- ident(a) -- becomes -- <expression> -- <simple-expression> -- <term> -- <factor> -- intcon(5) -- semicolon -- <assignment-statement> -- ident(b) -- becomes -- <expression> -- <simple-expression> -- <term> -- <factor> -- ident(a) -- plus -- <term></pre>


```

|-- beginsy
|-- <statement-list>
| |-- <assignment-statement>
| | |-- ident(x)
| | |-- becomes
| | |-- <expression>
| | | |-- <simple-expression>
| | | | |-- <term>
| | | | | |-- <factor>
| | | | | |-- intcon(10)
|-- semicolon
|-- <if-statement>
| |-- ifsy
| |-- <expression>
| | |-- <simple-expression>
| | | |-- <term>
| | | | |-- <factor>
| | | | | |-- ident(x)
| |-- gtr
| |-- <simple-expression>
| | |-- <term>
| | | |-- <factor>
| | | | |-- intcon(5)
|-- thensy
|-- <assignment-statement>
| |-- ident(y)
| |-- becomes
| |-- <expression>
| | |-- <simple-expression>
| | | |-- <term>
| | | | |-- <factor>
| | | | | |-- intcon(1)
|-- elsesy
|-- <assignment-statement>
| |-- ident(y)
| |-- becomes
| |-- <expression>
| | |-- <simple-expression>
| | | |-- <term>
| | | | |-- <factor>
| | | | | |-- intcon(0)
|-- semicolon
|-- <while-statement>
| |-- whilesy
| |-- <expression>
| | |-- <simple-expression>

```



```
program ArrayDemo;
```

```
const
```

```
  MAX == 10;
```

```
type
```

```
  Range == 1..10;
```

```
var
```

```
  arr: array[1..10] of integer;
```

```
  i: integer;
```

```
begin
```

```
  for i := 1 to MAX do
```

```
    arr[i] := i * 2;
```

```
end.
```

```
<program>
```

```
| |-- <program-header>
```

```
| |-- programsy
```

```
| |-- ident(ArrayDemo)
```

```
| |-- semicolon
```

```
| |-- <declaration-part>
```

```
| |-- <const-declaration>
```

```
| | |-- constsy
```

```
| | |-- ident(MAX)
```

```
| | |-- eql
```

```
| | |-- <constant>
```

```
| | | |-- intcon(10)
```

```
| | |-- semicolon
```

```
| |-- <type-declaration>
```

```
| | |-- typesy
```

```
| | |-- ident(Range)
```

```
| | |-- eql
```

```
| | |-- <type>
```

```
| | | |-- <range>
```

```
| | | | |-- <constant>
```

```
| | | | | |-- intcon(1)
```

```
| | | | | |-- period
```

```
| | | | | |-- period
```

```
| | | | | |-- <constant>
```

```
| | | | | | |-- intcon(10)
```

```
| | | |-- semicolon
```

```
| |-- <var-declaration>
```

```
| | |-- varsy
```

```
| | |-- <identifier-list>
```

```
| | | |-- ident(arr)
```

```
| | |-- colon
```

```
| | |-- <type>
```

```
| | | |-- <array-type>
```

```
| | | | |-- arraysy
```

```
| | | | |-- lbrack
```

```
| | | | |-- <range>
```

```
| | | | | |-- <constant>
```

```
| | | | | | |-- intcon(1)
```

```
| | | | | |-- period
```

```
| | | | | |-- period
```

```
| | | | | |-- <constant>
```

```
| | | | | | |-- intcon(10)
```

```
| | | | |-- rbrack
```

```
| | | |-- ofsy
```

```
| | |-- <type>
```

```
| | | |-- ident(integer)
```

```

|-- semicolon
|-- <identifier-list>
| |-- ident(i)
|-- colon
|-- <type>
| |-- ident(integer)
| |-- semicolon
|-- <compound-statement>
|-- beginsy
|-- <statement-list>
| |-- <for-statement>
| | |-- forsy
| | |-- ident(i)
| | |-- becomes
| | |-- <expression>
| | | |-- <simple-expression>
| | | | |-- <term>
| | | | | |-- <factor>
| | | | | | |-- intcon(1)
| | |-- tosy
| | |-- <expression>
| | | |-- <simple-expression>
| | | | |-- <term>
| | | | | |-- <factor>
| | | | | | |-- ident(MAX)
| | |-- dosy
| | |-- <assignment-statement>
| | | |-- <variable>
| | | | |-- ident(arr)
| | | | |-- <component-variable>
| | | | | |-- lbrack
| | | | | |-- <index-list>
| | | | | | |-- ident(i)
| | | | | | |-- rbrack
| | |-- becomes
| | |-- <expression>
| | | |-- <simple-expression>
| | | | |-- <term>
| | | | | |-- <factor>
| | | | | | |-- ident(i)
| | | | | | |-- times
| | | | | | |-- <factor>
| | | | | | | |-- intcon(2)
| |-- semicolon
|-- endsy
|-- period

```

3.4. Test Case 4

Input	Output
<pre> program SubprogramDemo; var result: integer; procedure printMsg(msg: array[char] of integer); var i: integer; begin writeln(msg); end; function add(a, b: integer): integer; begin add := a + b; end; begin result := add(3, 7); printMsg('Done'); end. </pre>	<pre> <program> -- <program-header> -- programsy -- ident(SubprogramDemo) -- semicolon -- <declaration-part> -- <var-declaration> -- varsy -- <identifier-list> -- ident(result) -- colon -- <type> -- ident(integer) -- semicolon -- <subprogram-declaration> -- <procedure-declaration> -- proceduresy -- ident(printMsg) -- <formal-parameter-list> -- lparent -- <parameter-group> -- <identifier-list> -- ident(msg) -- colon -- <array-type> -- arraysy -- lbrack -- ident(char) -- rbrack -- ofsy -- <type> -- ident(integer) -- rparent -- semicolon -- <block> -- <declaration-part> -- <var-declaration> -- varsy -- <identifier-list> -- ident(i) </pre>

```

|-- colon
|-- <type>
|-- ident(integer)
|-- semicolon
|-- <compound-statement>
|-- beginsy
|-- <statement-list>
|-- <procedure/function-call>
|-- ident(writeln)
|-- lparent
|-- <parameter-list>
|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- ident(msg)
|-- rparent
|-- semicolon
|-- endsy
|-- semicolon
|-- <subprogram-declaration>
|-- <function-declaration>
|-- functionsy
|-- ident(add)
|-- <formal-parameter-list>
|-- lparent
|-- <parameter-group>
|-- <identifier-list>
|-- ident(a)
|-- comma
|-- ident(b)
|-- colon
|-- ident(integer)
|-- rparent
|-- colon
|-- ident(integer)
|-- semicolon
|-- <block>
|-- <declaration-part>
|-- <compound-statement>
|-- beginsy
|-- <statement-list>
|-- <assignment-statement>
|-- ident(add)
|-- becomes
|-- <expression>

```

```

|-- <simple-expression>
|-- <term>
|   |-- <factor>
|   |   |-- ident(a)
|   |   |-- plus
|   |   |-- <term>
|   |   |   |-- <factor>
|   |   |   |   |-- ident(b)
|   |   |-- semicolon
|   |-- endsy
|-- semicolon
-- <compound-statement>
|-- beginsy
|-- <statement-list>
|   |-- <assignment-statement>
|   |   |-- ident(result)
|   |   |-- becomes
|   |   |-- <expression>
|   |   |   |-- <simple-expression>
|   |   |   |   |-- <term>
|   |   |   |   |   |-- <factor>
|   |   |   |   |   |   |-- <procedure/function-call>
|   |   |   |   |   |   |   |-- ident(add)
|   |   |   |   |   |   |   |-- lparent
|   |   |   |   |   |   |   |-- <parameter-list>
|   |   |   |   |   |   |   |-- <expression>
|   |   |   |   |   |   |   |--
|   |   |-- <simple-expression>
|   |   |   |-- <term>
|   |   |   |   |-- <factor>
|   |   |   |   |   |-- intcon(3)
|   |   |   |   |-- comma
|   |   |   |-- <expression>
|   |   |   |   |--
|-- <simple-expression>
|   |-- <term>
|   |   |-- <factor>
|   |   |   |-- intcon(7)
|   |   |-- rparent
|-- semicolon
-- <procedure/function-call>
|-- ident(printMsg)
|-- lparent
|-- <parameter-list>
|-- <expression>
|   |-- <simple-expression>

```



```

|-- <field-part>
|-- <identifier-list>
|   |-- ident(name)
|   |-- colon
|   |-- <type>
|       |-- ident(integer)
|-- semicolon
|-- <field-part>
|   |-- <identifier-list>
|       |-- ident(grade)
|       |-- colon
|       |-- <type>
|           |-- ident(integer)
|-- semicolon
|-- endsy
|-- semicolon
-- <var-declaration>
|   |-- varsy
|   |-- <identifier-list>
|       |-- ident(s)
|       |-- colon
|       |-- <type>
|           |-- ident(Student)
|-- semicolon
|-- <identifier-list>
|   |-- ident(score)
|-- colon
|-- <type>
|   |-- ident(integer)
|-- semicolon
-- <compound-statement>
|-- beginsy
|-- <statement-list>
|   |-- <assignment-statement>
|       |-- ident(score)
|       |-- becomes
|       |-- <expression>
|           |-- <simple-expression>
|               |-- <term>
|                   |-- <factor>
|                       |-- intcon(85)
|-- semicolon
|-- <case-statement>
|   |-- casesy
|   |-- <expression>
|       |-- <simple-expression>

```



```
program Hello
var
  a, b: integer;
begin
  a := 5;
end.
```

```
=== Parse Tree ===
<program>
| |-- <program-header>
| | |-- programsy
| | |-- ident(Hello)
```

[SYNTAX ERROR] Unexpected token 'varsy',
expected 'semicolon'

BAB IV

Penutup

4.1. Kesimpulan

Tugas besar Milestone 2 ini telah berhasil merancang dan mengimplementasikan tahap *Syntax Analysis* (Parser) dalam proses kompilasi. Program parser dibangun menggunakan bahasa pemrograman C++17 dengan mengimplementasikan algoritma *Recursive Descent*. Penggunaan algoritma ini dinilai sangat sesuai karena aturan grammar dapat dipetakan secara 1-ke-1 dan bahasa Arion tidak memerlukan *lookahead* lebih dari 1 token. Dalam pelaksanaannya, parser secara efektif menerima masukan token dari *Lexer*, memvalidasi kesesuaian susunannya terhadap grammar, lalu membangun representasi struktur sintaksis dalam bentuk *Parse Tree*. Melalui lima skenario pengujian (*test case*) yang telah dieksekusi, program terbukti berhasil memproses berbagai bentuk hierarki program secara akurat, mulai dari deklarasi variabel, percabangan *control flow*, perulangan, struktur data array dan *record*, hingga pemanggilan fungsi dan prosedur.

4.2. Saran

Saat ini, program kita masih mengandalkan file teks (seperti `token_output.txt`) sebagai "jembatan" antara *Lexer* dan *Parser*. Jadi, setelah *Lexer* selesai memproses kode, hasilnya disimpan dulu ke dalam file. Setelah itu, *Parser* harus membuka file tersebut, membaca isinya baris demi baris, dan menyusunnya kembali untuk diproses. Proses bolak-balik menulis dan membaca file ini sebenarnya cukup membebani kinerja. Ke depannya, disarankan agar *Lexer* langsung saja "mengoper" data hasil kerjanya ke *Parser* secara langsung di dalam program, tanpa perlu mampir disimpan ke file teks terlebih dahulu. Langkah ini dapat membuat proses kompilasi berjalan jauh lebih cepat dan efisien.

LAMPIRAN

Link Repository Github: <https://github.com/Zannn210/WOK-Tubes-IF2224-2026>

Tabel Pembagian Tugas

No	Tugas	Anggota	Persentase
1	- Mengimplementasikan prioritas ekspresi: <code><expression></code>, <code><simple-expression></code>, <code><term></code>, dan <code><factor></code>. - Menangani pembacaan operator: <code><relational-operator></code>, <code><additive-operator></code>, dan <code><multiplicative-operator></code>. - Mengembangkan logika untuk memformat keluaran Parse Tree agar ter-print ke terminal dan tersimpan ke file keluaran .txt.	Fauzan Mohamad Abdul Ghani (13524113)	20%
2	- Mengimplementasikan aturan produksi untuk <code><const-declaration></code> dan <code><constant></code>. - Mengimplementasikan pengenalan <code><type-declaration></code>, <code><type></code>, <code><array-type></code>, <code><range></code>, <code><enumerated></code>, dan <code><record-type></code>. - Mengimplementasikan aturan <code><var-declaration></code> dan <code><identifier-list></code>.	Safira Berlianti (13524128)	20%
3	- Mengimplementasikan pembuatan fungsi/prosedur: <code><subprogram-declaration></code>, <code><procedure-declaration></code>, <code><function-declaration></code>, <code><formal-parameter-list></code>, dan <code><parameter-group></code>. - Mengimplementasikan pemanggilan pemanggilan: <code><procedure/function-call></code> dan <code><parameter-list></code>. - Mengimplementasikan penugasan nilai: <code><assignment-statement></code>, <code><variable></code>, <code><component-variable></code>, dan <code><index-list></code>.	Eduard Daniel Ariajaya (13524129)	20%
4	- Mengimplementasikan pernyataan: <code><statement-list></code> dan <code><statement></code>. - Mengimplementasikan logika percabangan: <code><if-statement></code>, <code><case-statement></code>, dan <code><case-block></code>. - Mengimplementasikan logika perulangan: <code><while-statement></code>, <code><repeat-statement></code>, dan <code><for-statement></code>.	Muhammad Daffa Arrizki Yanma (13524133)	20%
5	- Memperbaiki pembacaan separator (space, newline, tab) pada DFA Lexer sebelumnya. - Membuat fungsi pembacaan file .txt yang berisi daftar token masukan.	Reysya Syafitri Mulya Ramadhan (13524137)	20%

	• Mengimplementasikan Recursive Descent untuk node fondasi: <program>, <program-header>, dan penggabungan dengan <compound-statement>.		
--	--	--	--

REFERENSI

<https://www.geeksforgeeks.org/compiler-design/recursive-descent-parser/>

<https://www.geeksforgeeks.org/compiler-design/classification-of-top-down-parsers/>

<https://ruslanspivak.com/lrbasi-part7>

https://textx.github.io/Arpeggio/latest/images/calc_parse_tree.dot.png